

COMPILER DESIGN

UNIT-4

[CS-603]

CODE GENERATION

Intermediate Code generation : Declarations
Assignment statements, Boolean expressions,
case statements, Back patching, Procedure
calls Code Generation : Issues in the
design of code generator, Basic Block &
flow graphs, Register allocation and
assignment, DAG representation of basic block
peephole optimization, generating code from
DAG.

INTRODUCTION TO CODE GENERATION

Code Generation is the final phase of compiler
Design.

It converts the intermediate code into target
machine code or assembly code.

Goals of Code Generator

- Generate correct code
- Generate efficient code
- Reduce memory usage
- Reduce execution time

Input to Code Generator

- Intermediate Code
- Symbol table
- Error information

Output

- Machine Code / Assembly Code.

4 Intermediate Code Generation

Intermediate Code is a machine-independent representation of source code.

Advantages -

- Easy optimization
- Portable
- Easy translation to machine code.

Types of Intermediate Code -

(a) Three Address Code → Each instruction contains at most 3 addresses

Ex:- $a = b + c * d$

TAC:- $t_1 = c * d$

$t_2 = b + t_1$

$a = t_2$

(b) **Quadruples** $\rightarrow$ Format (op, arg1, arg2, result)

Ex:-

op	arg1	arg2	result
*	c	d	t1
+	b	t1	t2
=	12	-	a.

(c) **Triples** $\rightarrow$ No temporary variables names are used.

Index	op	arg1	arg2
0	*	c	d
1	+	b	(0)
2	=	(1)	a

TRANSLATION OF DECLARATIONS

Compiler allocates memory for variables.

Ex:-
int a;
float b;

Symbol Table Entry	Name	Type	Address
	a	int	1000
	b	float	1004

ASSIGNMENT STATEMENTS

Assignment statements are converted into intermediate code.

Ex:- $x = a + b$

TAC:- $t1 = a + b$
 $x = t1.$

BOOLEAN EXPRESSIONS

A Boolean Expression is an expression that gives only two values.

- True (1)
- False (0)

It is mainly used in:

- If statements
- Loops
- Decision making

Ex:- $a > b$

If a is greater than b , result is TRUE, otherwise false.

Operators $\rightarrow$ $>$, $<$, $==$, $!=$, $\&\&$, $!$

$\downarrow$ greater than $\downarrow$ Equal to $\downarrow$ AND

$\downarrow$ less than $\downarrow$ Not Equal To $\downarrow$ NOT

In Compiler Design

Boolean expressions are translated into jump statements.

Example:-

If ($a < b$)

L1 $\rightarrow$ True Condition

Intermediate Code:-

L2 $\rightarrow$ False Condition

If $a < b$ goto L1
goto L2

BACKPATCHING [June 2025] [May 2022]

Backpatching is a technique used in compiler design to fill the missing jump addresses in intermediate code at a later stage.

While generating intermediate code for:

- If
- If-else
- While
- Boolean Expressions

The compiler may not know the exact target address immediately.

So, it leaves the address blank temporarily and fills it later.

This process is called Backpatching.

Need of Backpatching

During compilation:

- The compiler generates code line by line.
- Sometimes destination labels are not yet generated.
- Therefore jump addresses cannot be filled immediately.

So compiler:

1. Leaves blank spaces
2. Stores those instruction numbers
3. Fills correct addresses later.

Ex:- Source Code

```
if (a < b)
    x = y + z;
```

Intermediate code Initially

```
100: if a < b goto ____
101: goto ____
102: x = y + z.
```

- At this time compiler doesn't know where to jump.

After Backpatching -

```
100: if a < b goto 102
```

```
101: goto 103
```

```
102: x = y + z
```

```
103:
```

[Now the missing addresses are filled]

Application of Backpatching

Backpatching is mainly used in:

- Boolean expressions
- If statements
- If-else statements
- while loops
- for loops
- switch case statements.

Types of list Used in Backpatching

Compiler maintains special lists.

1. Truelist

Contains addresses of jump when condition become TRUE.

Ex:- If $a < b$ goto —

This instruction address is stored in truelist.

2. Falselist

Contains addresses of jumps when condition becomes FALSE.

Ex:- goto —

3. Nextlist

Contains addresses of statements that should execute next.

Used in loops & sequential statements.

Backpatching for Boolean Expressions

$a < b$

100: If $a < b$ goto —

101: goto —

Here:

100 $\rightarrow$ truelist

101 $\rightarrow$ falselist.

Later addresses are filled

Advantages of Backpatching -

Advantage	Description
• Simple code generation	→ Easy handling of jumps
• Useful for one-pass compilers.	→ No need for multiple scans.
• Efficient	→ Faster translation
• Handles forward jumps	→ Important in loops & conditions

Disadvantages

	Description
• Extra memory needed	→ For maintaining lists
• Complex Implementation	→ Compared to simple translation.

PROCEDURE CALL IN CODE GENERATION

A procedure call means calling a function or method in a program.

In code-generation, the compiler converts the function call into machine-level instructions

In Simple -

When one part of a program asks another function to do some work, it is called a procedure call.

Steps in Procedure Call -

- 1) **Pass Arguments**
Values are sent to the function.
- 2) **Save Return Address**
Compiler stores the address where execution should return after function finishes.
- 3) **Transfer Control**
Program jumps to the called function.
- 4) **Execute Function**
Function performs its task.
- 5) **Return Control**
After completion, control comes back to the main program.

Advantages-

- Code reuse
- Reduces program size
- Makes program modular
- Easy debugging.

ISSUES IN DESIGN OF CODE GENERATOR

Issues in design of code generator are the important factors considered while converting intermediate code into machine code to produce efficient and correct output.

main issue -

- 1) Instruction selection → Choosing the best machine instruction for an operation.
- 2) Register Allocation → Deciding which variables should be stored in CPU registers.
- 3) Register Assignment → Assigning variables to specific registers.
- 4) Order of Evaluation → Deciding the sequence of calculations in expressions.
- 5) Memory management → Using memory efficiently during code generation.
- 6) Target machine Features → Generating code according to the target computer architecture.

BASIC BLOCK [June 2025] [IMP]

Basic Block is a sequence of three Address Code Instructions in which flow of control enters at the beginning & leaves at the end.

This means:

- Control enters from the beginning only.
- Control leaves at the end only.
- No Jump occurs inside the block.

Begin (Single Entry)



$$t_1 = a * 5$$

$$t_2 = t_1 + 7$$

$$t_3 = t_2 - 5$$

$$t_4 = t_1 + t_3$$

$$x = t_4$$



End

(Single Exit)

Characteristics -

1. Execution starts from first statement only.
2. Execution ends at last statement only.

3. No branching inside the Block.
4. Statements executes sequentially.

Uses of Basic Block -

- Program optimization
- Flow graph construction
- Code generation
- Detecting redundant expressions.

Flow Graphs [June 2025] [May 2023]

A flow graph is a graphical representation of the flow of control between basic blocks in a program.

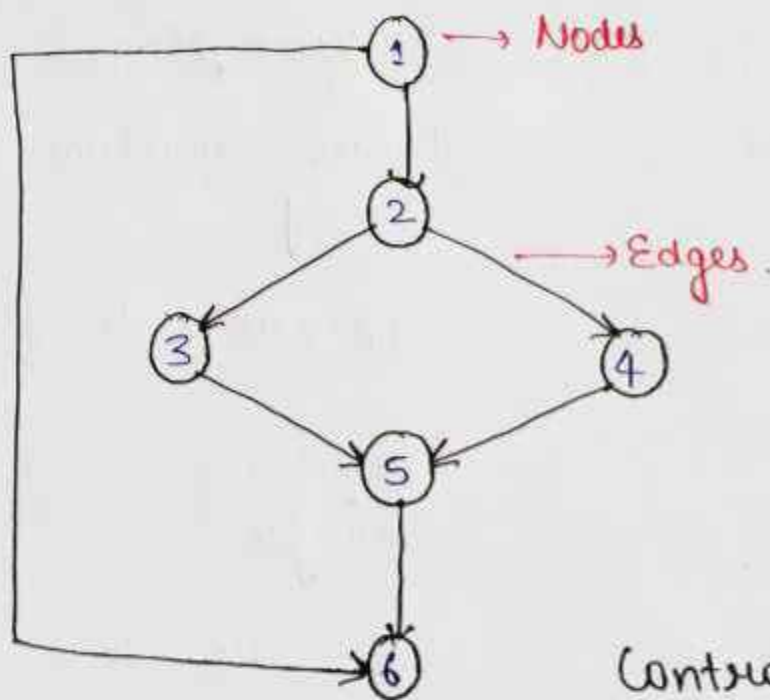
It shows:

- how the program moves from one block to another.
- the execution path of the program.

Flow graph is also called : Control Flow Graph (CFG).

Components of Flow Graph -

- 1) Nodes → Each node represent a basic block.
- 2) Edges → Edges represent the flow of control from one block to another.



Control Flow Graph.

Construction of Flow Graph - [June 2025]

- Divide program into basic blocks.
- Create one node for each basic block
- Connect nodes using edges according to program flow.

B1

```

w = 0;
x = x + y;
y = 0;
if (x > z)

```

B2

```

y = x;
x++;

```

B3

```

y = z;
x++;

```

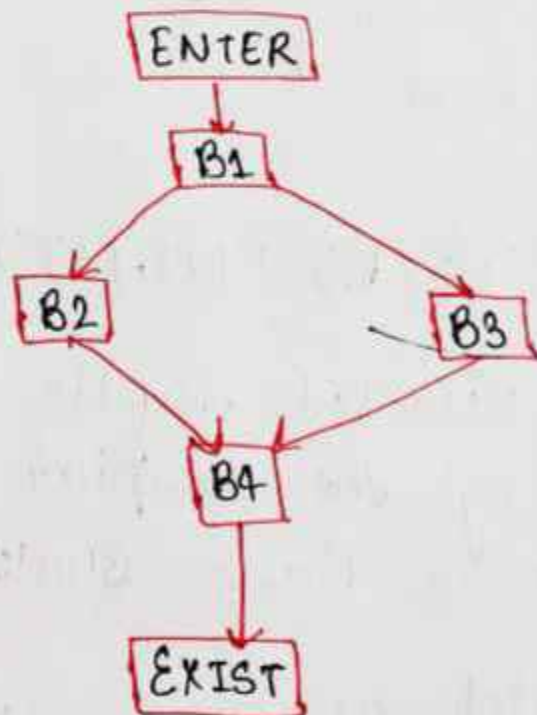
[Basic Blocks]

B4

```

w = x + z;

```



[Flow Graph]

Advantage

- Easy visualization
- Helps optimization
- Detects loops
- Finds unreachable code

Description

- Shows execution flow clearly.
- Improves code efficiency
- Useful in compiler analysis.
- Removes dead code.

Disadvantage

- Complex for large programs
- Extra memory required.

Description

- Large graphs becomes difficult.
- Stores nodes and edges.

[MIMP]

DAG REPRESENTATION OF BASIC BLOCK

DAG (Directed Acyclic Graph) is a graph used by the compiler to represent expressions inside a Basic Block. [May 2024] [May 2023]

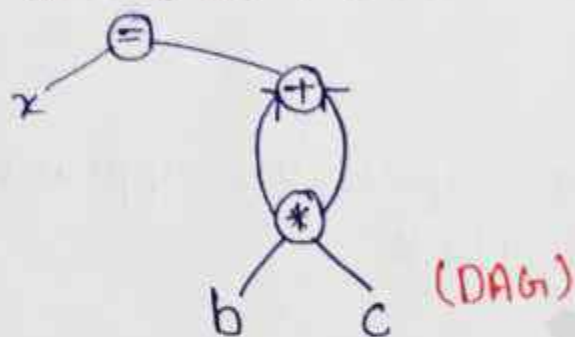
Directed → Arrows show the direction of computation.

Acyclic $\rightarrow$ No cycles or loops are present.

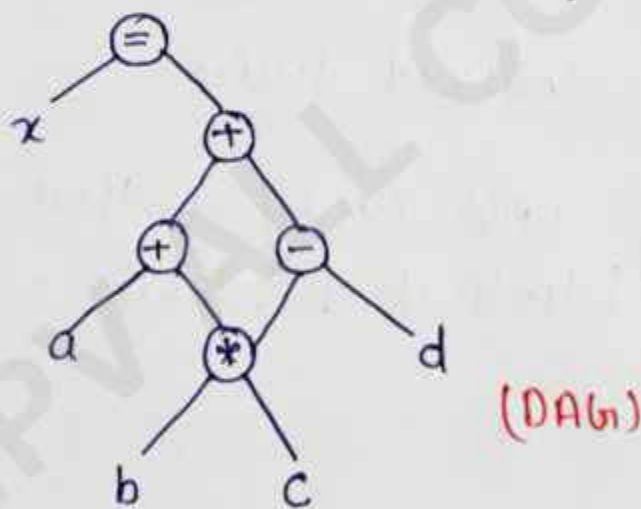
DAG helps the compiler:

- Remove repeated calculations.
- Optimize codes.
- Reduce execution time.

Example:- $x = b * c + b * c$



$x = (a + b * c) + (b * c - d)$



Components:

1. Leaf Nodes $\rightarrow$ Leaf Node represent: Variables
Constants

Ex:- a, b, x, x

2. Interior Nodes $\rightarrow$ Interior nodes represent: (+, -, *, /)

Ex:-  Means $\rightarrow b * c$.

Steps to Create a DAG

- 1) Create leaf nodes for variables & constants.
- 2) Create operator nodes for expressions.
- 3) If the same expressions appears again, reuse the existing node.
- 4) Attach variables names to the final result node.

Need of DAG

Sometimes the same expression appears multiple times in a basic block.

Example:- $a = b + c;$
 $d = b + c;$

Here, $b + c$ is calculated twice.

A compiler using DAG recognize that both expressions are identical & stores only once computation.

Thus:

- Execution becomes faster
- Less memory is used
- Optimize code is generated.

Advantages -

- i) Eliminate common subexpressions
- ii) Reduces Computation Time.
- iii) Saves memory
- iv) Improve Code Optimization.

Applications of DAB -

DAB is used in:

- Compiler optimization
- Intermediate code generation
- Common subexpression elimination
- Dead code elimination.
- Efficient machine code generation.

REGISTER ALLOCATION & ASSIGNMENT ^[MIMP] ^[June 2025]

When a Program is translated into machine code, many variables & intermediate results are created. The CPU performs operation using registers, which are very small & very fast storage locations inside the processor.

Since the number of registers is limited, the compiler must decide:

1. Which value should be kept in registers?

→ Register Allocation

2. Which specific register should be used?

→ Register Assignment.

These two tasks are important parts of code generation in compiler design.

Register

A register is a small memory location inside the CPU used to store data temporarily during execution.

Ex:- Suppose:- $C = a + b;$

The CPU may execute it as:

$$R_1 = a$$

$$R_2 = b$$

$$R_3 = R_1 + R_2$$

$$C = R_3$$

Here, R_1 , R_2 & R_3 are registers.

Registers are much faster than main memory.

REGISTER ALLOCATION [May 2023, June 2025]

Register Allocation is the process of deciding which variables & temporary values should be stored in registers during program execution.

In Simple-

Register allocation means choosing the values that will be kept in CPU registers to make execution faster.

Need of Register Allocation -

A program may contain many variables:

a, b, c, d, e, f, g.

But the CPU may have only a few registers:

R1, R2, R3, R4

Since registers are limited, the compiler must decide which variables deserve a register.

Example:-

$x = a + b;$

$y = x * c;$

$z = y + d;$

The compiler may keep:

x in R1

y in R2

Instead of storing them in memory repeatedly.

This improves speed.

Goal of Register Allocation -

- Minimize memory access.
- Use registers efficiently.
- Improve execution speed.
- Generate optimized code.

Benefits of Register Allocation

- 1) **Faster Execution** → Register access is much faster than memory access.
- 2) **Reduced Memory Traffic** → Less reading and writing from memory.
- 3) **Better Performance** → Programs runs more efficiently.

REGISTER ASSIGNMENT

Register Assignment is the process of assigning a specific register to each variable or temporary value selected during register allocation.

It means deciding exactly which register (R1, R2, R3 etc.) will store a particular Variable.

Example:-

Suppose allocation decides:

a, b and x should be stored in registers.

Assignment may be:

$a \rightarrow R1$

$b \rightarrow R2$

$x \rightarrow R3$

Now every value has a specific register.

Example of Allocation & Assignment

Statement - $x = a + b;$
 $y = x + c;$

Register Allocation -

Compiler decides:

a, b, x, y should use registers.

Register Assignment -

$a \rightarrow R1$

$b \rightarrow R2$

$x \rightarrow R3$

$c \rightarrow R4$

Generated Code:

$R3 = R1 + R2$

$R3 = R3 + R4$

$y = R3$

Challenges -

In Register Allocation:

- Registers are limited
- Many variables may be active at the same time.
- Compiler must choose wisely.

In Register Assignment:

- Avoid conflicts
- Ensure two active variables do not use the same register simultaneously.

Importance in Compiler Design

Register allocation & assignment:

- Improve program speed.
- Reduce memory access
- Generate efficient machine code.
- make better use of CPU resources.

PEEPHOLE OPTIMIZATION [May 2024] [May 2023]

Peephole Optimization is a code optimization technique in which the compiler looks at a small group of consecutive instructions (called a 'peephole') & replaces inefficient instructions with more efficient ones without changing the program's output.

Basically Peephole Optimization is a method used by a compiler to improve machine code by examining a small part of the code & replacing unnecessary or slow instruction with better ones.

Why it is called "Peephole"?

A peephole means a small hole through which we can see only a small portion of something.

Similarly, in peephole optimization, the compiler looks at only a small window of instructions at a time rather than the whole program.

Need for Peephole Optimization-

Sometimes the generated machine code contains:

- Unnecessary instructions
- Repeated operations
- Redundant Calculations
- Extra Jumps.

These instructions make the program slower & larger.

Peephole optimization removes such inefficiencies

Working of Peephole Optimization-

Step-1

Compiler selects a small sequence of instructions

Step-2

Checks whether they can be improved.

Step-3

Replaces them with shorter or faster instructions.

Step-4

Generates optimized code.

Types of Peephole Optimization -

1) Redundant Instruction Elimination:

Removes instruction that do not affect the result.

Before Optimization:

MOV R1, A

MOV A, R1

The second instruction is unnecessary.

After Optimization:

MOV R1, A

Result remains the same.

2) Constant Folding:

Performs calculations during compilation instead of execution.

Before:

$x = 5 + 10;$

Compiler computes:

$x = 15;$

Program executes faster.

3) Strength Reduction:

Replaces costly operations with simpler ones.

Before:

$x = y * z;$

After:

$x = y \ll 1;$

• Bit shift is faster than multiplication.

4) Null Sequence Elimination:

Removes instructions that have no effect.

Before:

ADD R1, 0

Adding zero changes nothing.

After:

(Instruction removed)

5) Flow of Control Optimization:

Removes unnecessary jumps.

Before:

IMP L1

L1:

Jump goes to the very next instructions.

After:

L1:

Jump is removed.

6) Algebraic Simplification:

Uses mathematical rules to simplify expressions.

Before:

$x = y + 0;$

$x = y * 1;$

After:

$x = y;$

$x = y;$

Example of Peephole Optimization -

Original Code

MOV R1, A

ADD R1, 0

MOV B, R1

Optimized Code

MOV R1, A

MOV B, R1

The instruction ~~add~~ ADD R1, 0 is removed because it has no effect.

Advantages of Peephole Optimization

- 1) Reduces Program Size → Unnecessary instructions are removed.
- 2) Increases Execution Speed → Fewer instructions need to be executed.
- 3) Improves Efficiency → Generated machine code becomes better.
- 4) Saves Memory → Less storage is required.
- 5) Simple to implement → Compiler only examines a small portion of code.

Disadvantages of Peephole Optimization

- 1) Limited Scope → It examines only a small sequence of instructions.
- 2) Cannot Perform Global Optimization → It may miss optimization opportunities in other parts of the program.

Applications of Peephole Optimization

- Compiler Design
 - Machine Code Optimization
 - Assembly Code Improvement
 - Embedded Systems
 - Performance Enhancement.
-